App Cibo — Design (v1 + fase 2 dispensa)

Data: 2026-10-12 Stato: in revisione

1. Obiettivo

Webapp (poi app iOS via Capacitor) per una persona o una famiglia che:

- genera con un'AI il **menù settimanale** in base a preferenze, allergie e presenze di tutti;
- produce la **lista della spesa** pensata per una spesa unica, separando ciò che scade troppo presto;
- permette di dire **"non mi piace"** a un piatto, lo sostituisce subito e **impara** (ingrediente o piatto);
- usa anche le **ricette dell'utente**;
- mostra **tempo medio**, **calorie** e **link alla ricetta originale** (link reale, non inventato);
- (fase 2) tiene una **dispensa automatica** con scadenze stimate, senza inserimento manuale.

Criterio di successo: la famiglia la usa ogni settimana per decidere i pasti e fare la spesa senza correzioni manuali frequenti, e nessun piatto proposto viola un'allergia di una persona presente.

Fuori ambito (spec future separate)

- Stima calorie da foto del piatto (modello vision su Workers AI + tabella nutrizionale).
- Input vocale con Whisper (Workers AI).
- Login Google / Apple (passo successivo al login email).
- Pubblicazione App Store (Capacitor).
- Più nuclei per utente.

2. Decisioni prese

Tema	Decisione
Piattaforma	Tutto su Cloudflare: un Worker (Hono) serve API + frontend statico (Workers Static Assets), account `lorenzo-dambrosio-2004`, piano Workers Paid
Frontend	React + Vite + TypeScript, PWA mobile-first, UI in italiano
Database	Cloudflare D1
Auth	Better Auth su D1, login con codice/link via email (invio con Resend). Google + Apple in seguito
AI	Livello `provider` intercambiabile: Claude API e Workers AI. Pagina di confronto per scegliere quale tenere
Ricette e link	L'AI genera la ricetta; il link viene da una ricerca web reale
Calorie	Calcolate da ingredienti × tabella nutrizionale in D1, non stimate dall'AI
Vincoli	Valgono solo per i presenti al pasto, **allergie comprese**; badge di avviso se un piatto contiene l'allergene di un assente
Colazione	Scelta per persona
Famiglia	Un utente appartiene a un solo nucleo; chi accetta un invito lascia il suo nucleo personale

3. Architettura

```
Browser / PWA (React)
        │  fetch /api/*
        ▼
Cloudflare Worker (Hono)
  ├─ /api/auth/*        Better Auth (sessioni in D1)
  ├─ /api/household/*   nucleo, membri, inviti
  ├─ /api/constraints/* vincoli alimentari
  ├─ /api/recipes/*     ricette proprie e generate
  ├─ /api/weeks/*       menù, pasti, assenze, rifiuti
  ├─ /api/shopping/*    lista spesa
  ├─ /api/pantry/*      dispensa (fase 2)
  └─ /api/admin/compare confronto provider
        │
        ├─ D1 (dati)
        ├─ Workers AI binding (Llama, in futuro Whisper/vision)
        ├─ Claude API (secret ANTHROPIC_API_KEY)
        ├─ Ricerca web (tool web_search di Claude / API di ricerca per Workers AI)
        └─ Resend (email di login)
```

Moduli (ognuno testabile da solo)

Modulo	Responsabilità	Dipende da
`domain/constraints`	Unione dei vincoli dei presenti, sinonimi allergeni, verifica di un piatto	nessuno (funzioni pure)
`domain/portions`	Scalare grammi e calorie sul numero di presenti	nessuno
`domain/nutrition`	Calorie da ingredienti × tabella	tabella passata in input
`domain/shopping`	Aggregazione lista, unità, reparti, "da comprare dopo"	tabella durate passata in input
`domain/learning`	Da un rifiuto ai vincoli/blocchi da creare; rilevamento rifiuti ripetuti	nessuno
`domain/pantry`	Ingresso dalla spesa, scalatura ai pasti, "scade presto"	nessuno
`ai/provider`	Interfaccia `generateWeek`, `replaceMeal`, `writeRecipe`, `findSourceUrl`, `mapIngredient`	implementazioni `claude`, `workersAi`
`ai/prompts`	Costruzione prompt e schemi JSON	nessuno
`db/*`	Query D1 tipizzate	D1
`routes/*`	HTTP: validazione input, autorizzazione, orchestrazione	moduli sopra

La logica di dominio è pura: riceve dati, restituisce dati. Le route fanno da collante.

4. Modello dati (D1)

Le tabelle di Better Auth (`user`, `session`, `verification`, …) sono gestite dalla libreria. Su `user` aggiungiamo `breakfast` (boolean, default false).

```
households(id, name, created_at)
household_members(household_id, user_id, role['owner'|'member'], joined_at)   -- user_id UNIQUE
household_invites(token, household_id, created_by, expires_at, used_at)

food_constraints(id, user_id, ingredient, level['allergy'|'never'|'little'|'dislike'],
                 source['manual'|'learned'], created_at)                      -- UNIQUE(user_id, ingredient)
blocked_dishes(user_id, dish_family, created_at)                              -- PK(user_id, dish_family)
favorites(user_id, recipe_id, created_at)                                     -- PK(user_id, recipe_id)

recipes(id, household_id, title, dish_family, minutes, servings,
        ingredients JSON [{name, grams}], steps TEXT NULL,
        source_url NULL, kind['own'|'own_ai'|'own_known'|'generated'],
        details_status['none'|'ready'], created_at)

weeks(id, household_id, start_date, shopping_day, status['draft'|'shopped'], provider, created_at)
meals(id, week_id, date, slot['breakfast'|'lunch'|'dinner'],
      recipe_id, speed['quick'|'long'], pinned BOOLEAN, status['planned'|'skipped'])
meal_absences(meal_id, user_id)                                               -- PK(meal_id, user_id)
recurring_absences(user_id, weekday 0–6, slot)

rejections(id, user_id, meal_id, recipe_id, reason['today'|'ingredient'|'dish'],
           ingredient NULL, created_at)

shopping_items(id, week_id, ingredient, grams, unit_label, aisle,
               buy_on DATE, frozen_hint BOOLEAN, manual BOOLEAN, checked BOOLEAN)

pantry_items(id, household_id, ingredient, grams, bought_on, expires_at, status['ok'|'used'|'trashed'])

ingredients_ref(name PK, kcal_100g, aisle, shelf_days, freezer_ok BOOLEAN)   -- seed + correggibile
ingredient_aliases(alias PK, name)                                            -- cache di mappatura AI
allergen_groups(group_name, ingredient)                                       -- es. frutta a guscio → noci
```

Ogni query sui dati del nucleo filtra per `household_id` dell'utente in sessione; vincoli, preferiti e rifiuti filtrano per `user_id`.

5. Account e nucleo

- Alla registrazione l'utente ottiene un nucleo personale (`owner`).
- L'owner genera un link d'invito (`household_invites`, scadenza 7 giorni, monouso).
- Chi accetta lascia il proprio nucleo (eliminato se vuoto) ed entra in quello nuovo. Vincoli, preferiti e rifiuti restano suoi; le ricette del vecchio nucleo vengono spostate nel nuovo.
- Un membro può uscire e tornare a un nucleo personale nuovo.

6. Generazione del menù

6.1 Impostazioni della settimana

- `shopping_day` (default sabato), inizio settimana = giorno della spesa.
- Per giorno: `quick` (≤ 25 min) o `long` (anche 60+). Default: quick lun–ven, long sab–dom.
- Pasti per giorno: pranzo e cena sempre; colazione se almeno un membro ha `breakfast = true`.
- Opzione "Riproponi preferiti".
- Pasti fissati: l'utente fissa un preferito a giorno+pasto (`pinned = true`), non viene generato.

6.2 Passo 1 — menù (una chiamata)

Input al provider:

- per ogni pasto da generare: data, slot, velocità, elenco presenti;
- per ogni presente: vincoli (`allergy` / `never` = vietato, `little` = poco, `dislike` = evita) e `blocked_dishes` ;
- titoli e `dish_family` dei pasti delle ultime 2 settimane (da non ripetere);
- elenco ricette del nucleo (id, titolo, minuti, ingredienti) — inserirne almeno 2 se disponibili;
- preferiti dei membri se l'opzione è attiva (esenti dalla regola anti-ripetizione).

Output (JSON validato con schema): per pasto `{date, slot, recipe_id?, title, dish_family, minutes, servings, ingredients[{name, grams}]}` . Nessun procedimento.

6.3 Controllo di sicurezza (codice nostro)

Per ogni pasto: normalizza gli ingredienti (`ingredient_aliases` , poi `mapIngredient` dell'AI se mancano, risultato salvato), espandi i gruppi allergeni, confronta con `allergy` e `never` dei presenti e con `blocked_dishes` . In caso di violazione si rigenera **solo quel pasto** con `replaceMeal` , al massimo 2 tentativi; se fallisce ancora, il pasto resta vuoto con il messaggio "Nessuna proposta valida, scegli tu".

Badge di avviso: se un piatto contiene un allergene di un membro **assente**, mostra "⚠️ contiene X, allergene di Nome (assente)".

6.4 Passo 2 — dettaglio ricetta (alla prima apertura)

Se `details_status = 'none'` :

1. `writeRecipe` → procedimento in italiano;
2. `findSourceUrl` → ricerca web reale " ricetta", primo risultato pertinente su un sito di ricette; si salva solo un URL presente nei risultati di ricerca, mai uno scritto dal modello;
3. salva su `recipes` , `details_status = 'ready'` .

Ricette `own_ai` : stesso flusso al salvataggio. `own_known` : solo titolo; gli ingredienti (per la spesa) li stima l'AI una volta e l'utente può correggerli; nessun procedimento né link.

6.5 Calorie

`kcal per porzione = Σ(grams × kcal_100g / 100) / servings` . Ingredienti senza valore → esclusi e segnalati con "calorie parziali".

6.6 Porzioni e presenze

- Default: tutti presenti. "Non ci sono" crea `meal_absences` ; `recurring_absences` vengono applicate alla creazione della settimana.
- Grammi mostrati = grammi ricetta × presenti / `servings` .
- Zero presenti → `status = 'skipped'` .
- I vincoli considerati sono solo quelli dei presenti. Se un'assenza viene tolta e il piatto viola i vincoli della persona che ritorna, il pasto viene segnalato e si propone la sostituzione.

7. “Non mi piace” e apprendimento

Solo un presente può rifiutare. Motivi:

Motivo	Effetto
`today`	Solo sostituzione
`ingredient` (uno o più, scelti tra gli ingredienti del piatto)	`food_constraints` `dislike` / `learned` per l’utente (non declassa vincoli esistenti più forti)
`dish`	`blocked_dishes` con la `dish_family` per l’utente

Poi `replaceMeal`: stessa velocità, vincoli aggiornati dei presenti, preferisce ingredienti già in lista (prima della spesa) o in dispensa (dopo la spesa). Il controllo di sicurezza del §6.3 si applica anche qui.

- Rifiuti `today` ripetuti: 3 rifiuti di piatti con lo stesso ingrediente in 30 giorni → domanda “Forse non ti piace X?” (sì → `dislike` learned).
- Impostazioni → “Cose che ho imparato su di te”: elenco dei `learned` e dei piatti bloccati, cancellabili; un `dislike` si può promuovere a `never`.

8. Lista della spesa

- Somma ingredienti dei pasti `planned` della settimana con porzioni scalate; accorpamento per nome normalizzato; etichetta leggibile (es. “Cipolle, 450 g (~3)”).
- Raggruppamento per `aisle`.
- **Da comprare dopo**: per ogni uso di un ingrediente, se `data_pasto − shopping_day > shelf_days`, quella quota va in una voce separata con `buy_on = data_pasto − 1` (minimo il giorno stesso); se `freezer_ok`, suggerimento “oppure surgelato”.
- Voci manuali aggiungibili.
- Spunte condivise; il client aggiorna la lista ogni 5 secondi mentre è aperta.
- Prima di “Spesa fatta” la lista si ricalcola a ogni modifica dei pasti, conservando voci manuali e spunte. Dopo, la lista è congelata.

9. Dispensa (fase 2)

- “Spesa fatta” → ogni voce spuntata entra in `pantry_items` con `expires_at = oggi + shelf_days`; `weeks.status = 'shopped'`.
- A fine giornata (Cron Trigger del Worker) i pasti passati scalano i loro ingredienti dalla dispensa (FIFO per scadenza).
- Correzioni manuali: “finito”, “buttato”, “ne ho ancora (grammi)”.
- Home: sezione “Scade presto” (entro 2 giorni).
- Sostituzioni e generazione della settimana successiva ricevono la lista di ingredienti in dispensa, con priorità a quelli in scadenza.
- `ingredients_ref.shelf_days`: seed iniziale di ~300 ingredienti comuni generato una volta con l’AI e rivisto; correggibile dall’app.

10. Provider AI

```
interface AiProvider {
  generateWeek(input: WeekRequest): Promise<WeekPlan>
  replaceMeal(input: ReplaceRequest): Promise<PlannedMeal>
  writeRecipe(input: RecipeRequest): Promise<{ steps: string }>
  findSourceUrl(title: string): Promise<string | null>
  mapIngredient(names: string[]): Promise<Record<string, string>>
}
```

- **Claude**: SDK `@anthropic-ai/sdk`. `generateWeek` con `claude-sonnet-5-5`; `replaceMeal`, `mapIngredient` con `claude-haiku-5-5`; `writeRecipe` con Sonnet; `findSourceUrl` con il tool `web_search`. Output JSON tramite structured outputs. I modelli sono configurabili da variabile d’ambiente.
- **Workers AI**: binding `AI`, modello Llama più recente disponibile; `findSourceUrl` tramite un’API di ricerca web esterna (scelta in fase di piano).
- Provider attivo per nucleo in configurazione (default Claude). Ogni risposta viene validata con lo schema (zod): se non valida, un nuovo tentativo; poi fallback sull’altro provider.
- **Confronto** (`/admin/compare`, accesso limitato all’email dell’admin): stessa `WeekRequest` a entrambi, risultati affiancati con latenza, token o costo stimato, numero di violazioni rilevate dal §6.3.

Costo stimato con Claude: ~0,30–0,60 € a settimana per nucleo.

11. Errori

- AI non raggiungibile o risposta invalida: un nuovo tentativo, poi l’altro provider, poi un messaggio all’utente (“Riprova tra poco”), senza perdere la settimana esistente.
- Ricerca link fallita: ricetta mostrata senza link, nuovo tentativo alla prossima apertura.
- Ingrediente non mappabile: niente calorie per quell’ingrediente, reparto “Altro”, `shelf_days` prudente (3).
- Il controllo allergie non dipende mai dall’AI.

12. Test

- Vitest con `@cloudflare/vitest-pool-workers`.
- Unit test sui moduli `domain/*`: unione vincoli dei presenti, gruppi allergeni, rilevamento violazioni, scalatura porzioni, calorie, aggregazione lista e unità, “da comprare dopo”, apprendimento dai rifiuti e rifiuti ripetuti, scalatura dispensa.
- Test delle route con D1 locale e provider AI finto che restituisce risposte registrate (incluse risposte con violazioni e JSON invalido).
- Nessuna chiamata AI reale nei test automatici; il confronto provider è manuale.

13. Interfaccia (schermate)

1. Login (email → codice).
2. Onboarding: nome, colazione sì/no, allergie e ingredienti vietati.
3. **Settimana**: griglia giorni × pasti; per pasto titolo, minuti, kcal, ⭐, “Non ci sono”, “Non mi piace”; impostazioni della settimana (giorno spesa, veloce/lunga per giorno, preferiti, pasti fissati); pulsante “Genera”.
4. **Ricetta**: ingredienti scalati, procedimento, link originale, kcal, badge allergeni.
5. **Spesa**: per reparto, sezione “Da comprare dopo”, spunte, “Spesa fatta”.
6. **Le mie ricette**: aggiungi (scritta / trova tu / la so io), modifica.
7. **Famiglia**: membri, invito, esci.
8. **Profilo**: vincoli, “Cose che ho imparato”, colazione, assenze ricorrenti.
9. **Dispensa** (fase 2): elenco, scade presto, correzioni.

14. Fasi di consegna

1. **Fondamenta**: progetto Worker + Vite, D1 e migrazioni, auth email, nucleo e inviti, profilo e vincoli.
2. **Menù**: provider (Claude + Workers AI), generazione, controllo sicurezza, assenze, preferiti, dettaglio ricetta con link, calorie.
3. **Non mi piace**: rifiuti, apprendimento, sostituzione.
4. **Spesa**: lista, reparti, “da comprare dopo”, spunte condivise.
5. **Confronto provider**.
6. **Dispensa** (fase 2).

Ogni fase è utilizzabile e deployata su `*.workers.dev` prima della successiva.